

PRACTICAL 4:

Creating and Managing a Virtual Machine.

- a. Set up a virtual machine on a cloud provider or locally (e.g., using VMWare or Azure).
- b. Configure networking concepts like virtual networks (Vnet), IP addresses, and ports
- c. Deploy the Flask/FastAPI app manually on the VM.

To create and manage a Virtual Machine using VMware, configure networking, and deploy a Flask application manually on Ubuntu.

Software Requirements

- VMware Workstation
 - Ubuntu (already installed)
 - Python 3
 - Flask
 - Terminal
 - Web Browser
-

Part (a): Set Up a Virtual Machine Using VMware

Step 1: Open VMware Workstation

1. Launch **VMware Workstation**.
 2. Select your **Ubuntu Virtual Machine**.
 3. Click **Power On**.
-

Step 2: Log in to Ubuntu

Enter your Ubuntu username and password.

Step 3: Update Ubuntu

Open the Terminal and run:

```
sudo apt update  
sudo apt upgrade -y
```

Step 4: Verify Python Installation

```
python3 --version
```

Example Output:

```
Python 3.12.3
```

If Python is not installed:

```
sudo apt install python3 python3-pip -y
```

Step 5: Install Flask

```
pip3 install flask
```

Verify:

```
pip3 show flask
```

Result (Part a)

The Ubuntu Virtual Machine is successfully created and configured using VMware.

Part (b): Configure Networking (Virtual Network, IP Address, and Ports)

Step 1: Check the IP Address

Run:

```
ip addr
```

or

```
hostname -I
```

Example Output:

```
192.168.43.128
```

This is your VM's IP address.

Step 2: Understand VMware Network Modes

Network Mode	Description
NAT	VM shares the host's internet connection (recommended for beginners).
Bridged	VM gets its own IP address on the same network as the host.
Host-only	Communication only between the host and the VM; no internet access.

Recommended: Use NAT unless your assignment specifies otherwise.

To check or change the mode:

1. Shut down the VM.
 2. In VMware, select the VM.
 3. Go to **VM** → **Settings** → **Network Adapter**.
 4. Choose **NAT** (or **Bridged**, if required).
-

Step 3: Verify Internet Connectivity

```
ping google.com
```

If you receive replies, the network is working.

Step 4: Check Listening Ports

```
sudo ss -tuln
```

or

```
netstat -tuln
```

Typical output:

```
tcp LISTEN 0 128 0.0.0.0:5000
```

Port **5000** is commonly used by Flask.

Step 5: Firewall (if enabled)

Allow port **5000**:

```
sudo ufw allow 5000
```

Check the firewall status:

```
sudo ufw status
```

Result (Part b)

The VM network is configured, the IP address is verified, and the required application port is available.

Part (c): Deploy the Flask Application Manually

Step 1: Create a Project Folder

```
mkdir FlaskApp  
cd FlaskApp
```

Step 2: Create `app.py`

```
from flask import Flask  
  
app = Flask(__name__)  
  
@app.route('/')  
def home():  
    return "Hello from Ubuntu Virtual Machine!"  
  
if __name__ == "__main__":  
    app.run(host="0.0.0.0", port=5000)
```

Step 3: Run the Application

```
python3 app.py
```

Expected output:

```
* Running on http://0.0.0.0:5000
```

Step 4: Test the Application

If you're using **NAT**, first test inside the VM:

Open Firefox in Ubuntu:

```
http://localhost:5000
```

Output:

```
Hello from Ubuntu Virtual Machine!
```

If you're using **Bridged** or have configured **NAT port forwarding**, you can also access it from the host using:

```
http://<VM_IP_Address>:5000
```

Example:

```
http://192.168.43.128:5000
```

Step 5: Stop the Flask Server

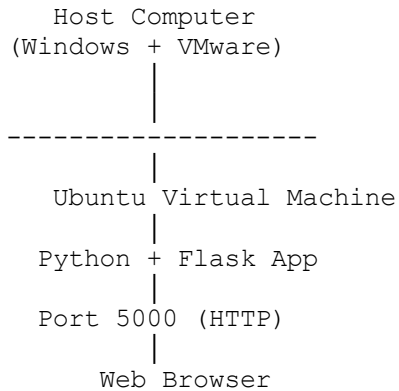
Press:

```
Ctrl + C
```

Project Structure

```
FlaskApp/  
└─ app.py
```

Architecture



Commands Summary

Command	Purpose
<code>sudo apt update</code>	Update package list
<code>python3 --version</code>	Check Python version
<code>pip3 install flask</code>	Install Flask
<code>hostname -I</code>	Display the VM's IP address
<code>ip addr</code>	Show network interfaces and IP addresses
<code>sudo ss -tln</code>	Show listening ports
<code>sudo ufw allow 5000</code>	Allow Flask through the firewall
<code>python3 app.py</code>	Start the Flask application
